

Project 1: Rotary Encoder Driver

Author: Eiríkur Bernharðsson

Board: Arduino Nano (ATmega328P, 16 MHz)

Motor: JGA12-N20 gearmotor, 1:100 gearbox, Hall effect quadrature encoder

Video: https://youtu.be/Dz__JQw9IRg

Repository: https://github.com/eirikur7/EmbeddedSystems_Project1

A note on the two versions I started writing this up and before I knew it, it was at 16 pages, so I made this shorter version. If you want more detail on any point, it is all in the longer version results_verbose

The git repository is private, but I sent you an invite to your RU email.

If you use another email for your git account then send me an email and I will send you a new invite.

Summary

	Calculated	Measured
Encoder resolution	1400 counts per revolution	about 1400 (turned by hand)
Maximum pulse rate	3617 edges/s (1808 Hz)	3604 edges/s (1802 Hz)
Must sample faster than (counting)	every 276 μ s	fails above 277 μ s
Must sample faster than (direction)	every 138 μ s	fails above about 139 μ s
Response time, polled	under 100.7 μ s	59 to 96 μ s
Response time, interrupt		2.2 to 7.9 μ s

1. Setup

Pin	Use
D2	Encoder C1, the counted channel. Also INT0, used in Part 3
D3	Encoder C2, read at each C1 edge for direction
D4	LED, pulsed once per handled state change. Scope probe point
D0, D1	Serial to the laptop, 9600 baud

All pins are on port D, which is what `Digital_in` and `Digital_out` from A1.2 use. Motor run from 6 V with no load. Scope: Rohde & Schwarz RTB2004, Ch1 on C1, Ch2 on C2, Ch3 on the LED. No timers are used; Parts 1 and 2 time their samples with `_delay_us()` .

2. Part 1: Polled driver

Pulse rate

7 pulses per turn of the motor shaft and a 1:100 gearbox give 700 cycles per output revolution. Both edges of each cycle are counted, so the resolution is **1400 counts per revolution**.

The datasheet gives 155 RPM with no load and 90 RPM at rated torque. No load is the right figure, since it is the fastest the shaft can turn. At 155 RPM, or 2.583 rev/s:

```
signal frequency = 700 x 2.583 = 1808 Hz (period 553 µs)
edge rate        = 1400 x 2.583 = 3617 edges/s
```

Each cycle holds two edges, so the edge rate is twice the frequency.

Sampling rate

Requirement	Max time between samples	Minimum rate
Do not miss a pulse	276 µs (half a period)	3.6 kHz
Get the direction right	138 µs (quarter period)	7.2 kHz

Counting needs one sample while C1 is high and one while it is low, so half a period. Direction comes from the phase between C1 and C2, and that pair changes state every quarter period. **Direction is the stricter requirement and sets the design.**

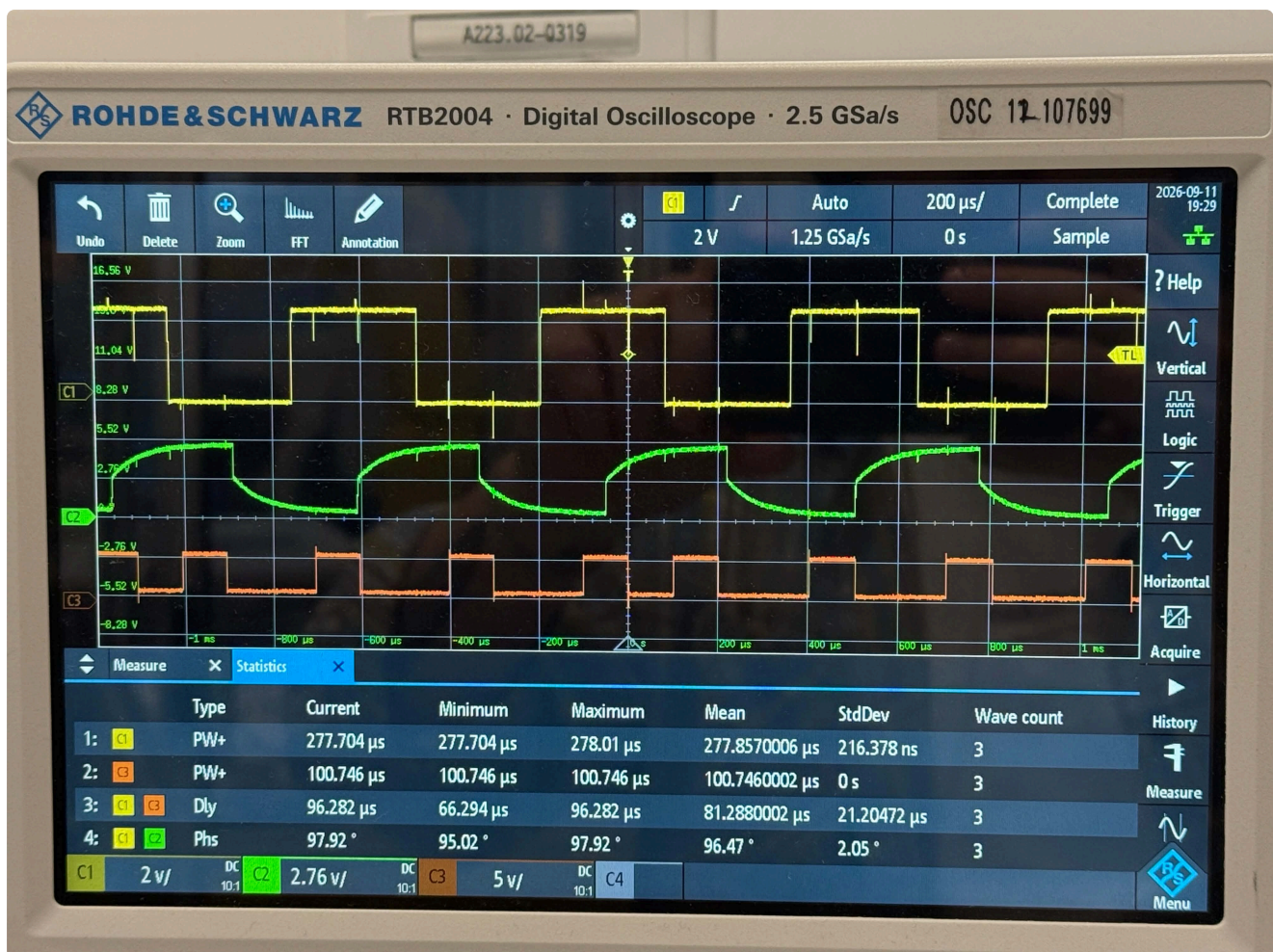
The loop samples with `_delay_us(100)` , about 10 kHz. That is 2.7x margin on counting and 1.4x on direction.

Implementation

`Encoder` holds two `Digital_in` objects and exposes `int position()` . `update()` checks C1 for a change and, when it finds one, reads C2 to decide whether to add or subtract. All pin access goes through `Digital_in` , so `encoder.cpp` has no registers in it and the header shows no hardware detail. The LED goes high for the one loop pass in which a change was handled.

Scope measurements

Measured	Value	Note
C1 positive pulse width	277.4 μ s	Predicted 276.5 μ s, so 0.3 % off
C1 to C2 phase	-88.5° / +97.9°	Sign flips when the motor leads are swapped
Loop period (LED width)	100.746 μ s	The 100 μ s delay plus 0.75 μ s of work
Response, C1 edge to LED	59 to 96 μ s	Always inside one loop period, as predicted



The response time varies because an edge can arrive anywhere inside a sample period.

Where the sampling fails

The loop delay was raised and the serial output recorded for 10 seconds at each setting.

Delay	Count rate	Result
100 μ s	3470 /s	Correct. 4 % low, because the periodic print stalls the loop
200 μ s	1400 /s	No edges missed, but past 138 μ s, so many count the wrong way

Delay	Count rate	Result
278 μ s	441 /s	At the edge spacing. Samples alias, the LED stays high, the count nearly stops

Both limits appear where predicted. A slow count does not mean lost pulses: every edge is still seen, but the wrong directions subtract from the right ones.

Verification

Running the motor one way makes the count climb and swapping the leads makes it fall, so the counter follows direction. Turning the marked output shaft one full revolution by hand gave roughly 1400 counts every time, which confirms $7 \times 100 \times 2$.

3. Part 2: Printing over UART

`print_interval` was changed from 2000 to 1, so every sample prints. Nothing else changed.

	Part 1	Part 2
Loop period	100.746 μ s	7.2895 ms
Count rate	3470 /s	roughly 20 /s



The serial output does interfere, and printing every sample destroys the count. The loop period is now just the time to send one line: 7 characters at 10 bit times each is 7.29 ms predicted against 7.2895 ms measured. The `_delay_us(100)` no longer matters.

That is 26 times the counting limit and 53 times the direction limit. The level of C2 at sampling time no longer relates to the C1 edge being seen, so the direction decision is close to random. One run wandered between 6 and 13 and never accumulated; another climbed about 200 counts in 10 seconds instead of 36 000.

The signal at the pin is unchanged and the driver is still correct. The problem is a blocking serial write inside the sampling loop. Printing every 2000 samples does not remove that cost, it only spreads it out, and it still costs about 4 % of the counts.

4. Part 3: Interrupts

Implementation

C1 is on D2, the INT0 pin. `init()` enables INT0 and triggers on any change, which keeps both edges and the 1400 resolution. Three details were needed:

- An interrupt handler cannot be a class member, so `init()` stores the instance in a file-scope pointer and `ISR(INT0_vect)` calls through it.
- `count` is written by the interrupt and read by the main loop, so it is `volatile`.

- count is 16 bit, so `position()` turns interrupts off across the read to avoid returning half of an old value.

The LED moved into the handler. It rises on entry and falls on exit, so the rising edge is the response time and the width is the handler run time. `main()` is now just a print loop with no delay.

Measurements



	Part 1	Part 2	Part 3
Sampling	polled, 100.7 μs	polled, 7.29 ms	interrupt
Printing	1 per 2000 samples	every sample	every sample
Response time	59 to 96 μs	meaningless	2.20 to 7.85 μs, mean 2.53
Handler run time			2.12 μs
Counts in 10 s	34 700	roughly 200	same order as Part 1

Part 3 runs the same serial load that destroyed the count in Part 2, and the count still accumulates normally. The interrupt separates the encoder read from whatever the main loop is doing.

Response time is the strongest evidence. Over 726 acquisitions it stays between 2.20 and 7.85 μs , three orders of magnitude inside both limits. At 2.12 μs per edge and 3604 edges per second the encoder costs about 0.8 % of the processor.

One surprise

The first attempt counted far too fast. Brush noise from the motor put sub-microsecond glitches on C1 during each low period and the interrupt fired on every one. Polling had ignored them by accident, since sampling every 100 μs almost never landed on one.

The noise was direction dependent, so running the motor the other way gave a clean signal and a correct count. That is a workaround, not a fix. The proper fix is filtering the input, for example an RC low pass on C1 with 1 k Ω and 10 nF.

5. Resource use

Build	Flash (of 30720 B)	RAM (of 2048 B)
Part 1	1450 B (4.7 %)	179 B (8.7 %)
Part 2	1440 B (4.7 %)	179 B (8.7 %)
Part 3	1584 B (5.2 %)	181 B (8.8 %)

Most of the RAM is the Arduino serial library's two 64-byte buffers. Part 3 is about 140 bytes larger: the interrupt vector, the register save and restore, and the interrupt safe read.

Both designs spend under 1 % of the processor on the encoder, but the polled version sits in `_delay_us()` the rest of the time and can do no other work. The interrupt version leaves about 99 % of the processor free.

6. Conclusion

The encoder gives 1400 counts per revolution and 3617 edges per second at 155 RPM, confirmed on the scope to within 0.3 %.

Two different timing limits follow: 276 μs not to miss a pulse, 138 μs to get the direction right. Direction is the stricter one, and the sweep confirms both. At 200 μs the counting still works while the direction is already wrong, and only past 277 μs does the counting itself break.

The polled driver at 100 μs meets both limits, but it occupies the processor full time and loses about 4 % of the counts to the print. Printing every sample in Part 2 stretches the loop by 72 times and destroys the count. The interrupt version fixes both: response time about 30 times better, the count survives the heaviest serial load, and the processor is almost free. Its one drawback is sensitivity to electrical noise that polling had been hiding.

Appendix: submitted files

Files are submitted individually rather than as an archive, so the per-part folders are flattened. Files that are identical in every part keep their name; the ones that differ carry a part suffix.

File	Used by
<code>digital_in.h</code> , <code>digital_in.cpp</code>	All parts, unchanged from A1.2
<code>digital_out.h</code> , <code>digital_out.cpp</code>	All parts, unchanged from A1.2
<code>encoder_part1_2.h</code> , <code>encoder_part1_2.cpp</code>	Parts 1 and 2, polled driver
<code>encoder_part3.h</code> , <code>encoder_part3.cpp</code>	Part 3, interrupt driver
<code>main_part1.cpp</code>	Part 1, prints every 2000 samples
<code>main_part2.cpp</code>	Part 2, prints every sample
<code>main_part3.cpp</code>	Part 3, prints continuously

Parts 1 and 2 differ only in the value of `print_interval` . In the repository the files sit in `results/part1/` , `results/part2/` and `results/part3/` under their plain names.